

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of	15%	Effectively	Applies	Limited	Unable to

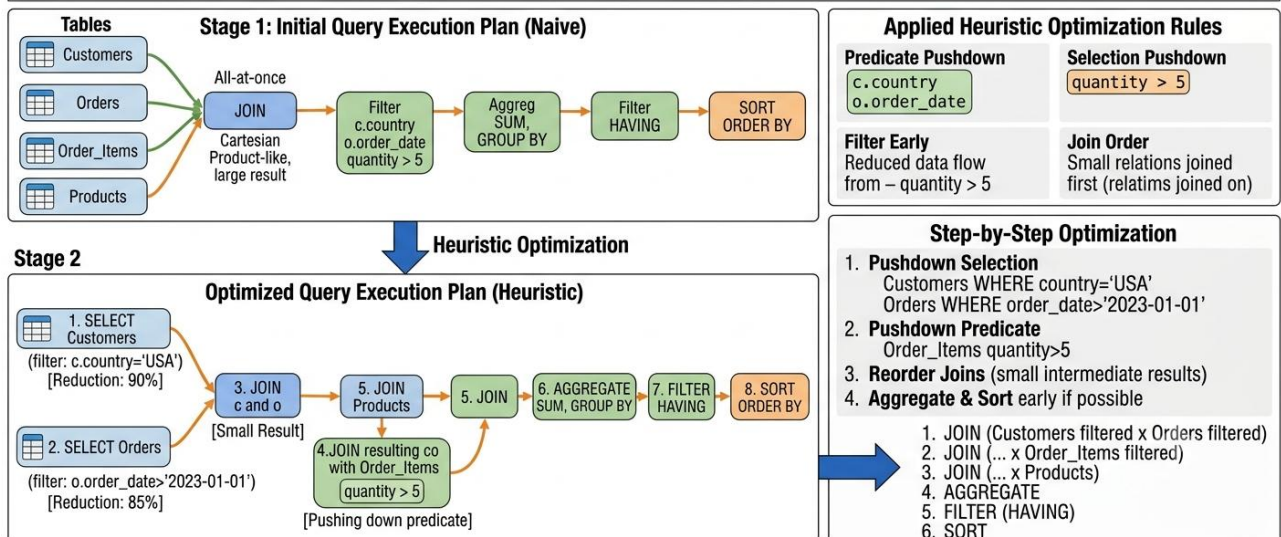
Concepts		applies appropriate concepts, methods, or tools	concepts with minor errors	application; requires guidance	apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step

```
mysql> SELECT AVG(E2.salary)
-> FROM Employee E2
-> WHERE E2.dept_id = E.dept_id
-> SELECT dept_id, AVG(salary) AS avg_salary
-> FROM Employee
-> GROUP BY dept_id;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'S
ELECT dept_id, AVG(salary) AS avg_salary
FROM Employee
GROUP BY dept_id' at line 4
mysql> SELECT E.emp_name,
-> E.salary,
-> P.project_name
-> FROM Employee E
-> JOIN (
-> SELECT dept_id, AVG(salary) AS avg_salary
-> FROM Employee
-> GROUP BY dept_id
-> ) A
-> ON E.dept_id = A.dept_id
-> JOIN (
-> SELECT dept_id
-> FROM Department
-> WHERE dept_name = 'IT'
-> ) D
-> ON E.dept_id = D.dept_id
-> JOIN Works_On W
-> ON E.emp_id = W.emp_id
-> JOIN (
-> SELECT project_id, project_name, dept_id
-> FROM Project
-> WHERE budget > 100000
-> ) P
-> ON W.project_id = P.project_id
-> WHERE E.salary > A.avg_salary;
+-----+-----+-----+
| emp_name | salary | project_name |
+-----+-----+-----+
| Arun     | 70000.00 | Database System |
+-----+-----+-----+
1 row in set (0.030 sec)

mysql>
```

```
SELECT c.customer_name, o.order_date, SUM(oi.quantity * p.price) AS total_order_amount
FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id
JOIN (SELECT order_id, product_id, quantity FROM Order_Items WHERE quantity > 5) oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
WHERE c.country = 'USA' AND o.order_date > '2023-01-01'
GROUP BY c.customer_name, o.order_date HAVING total_order_amount DESC;
```



2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes

Answer:

Number of pages:

$$B(R) = \frac{10000}{100} = 100 \text{ pages}$$
$$B(S) = \frac{5000}{100} = 50 \text{ pages}$$

1. Nested-Loop Join

Using S as the outer relation:

$$\begin{aligned} \text{Cost} &= B(S) + B(S) \times B(R) \\ &= 50 + (50 \times 100) \\ &= \boxed{5050 \text{ I/O}} \end{aligned}$$

2. Sort-Merge Join

For external sorting:

$$\begin{aligned} \text{Cost}(\text{sort}(R)) &\approx 2B(R) \\ \text{Cost}(\text{sort}(S)) &\approx 2B(S) \end{aligned}$$

Approximately:

$$\begin{aligned} &2(100) + 2(50) + 100 + 50 \\ &= \boxed{450 \text{ I/O}} \end{aligned}$$

The exact cost can vary depending on the number of merge passes.

3. Hash Join

Since S has only 50 pages and available memory is 50 pages, S can potentially be used as the build relation.

Approximate cost:

$$\begin{aligned} \text{Cost} &= 3(B(R) + B(S)) \\ &= 3(100 + 50) \\ &= \boxed{450 \text{ I/O}} \end{aligned}$$

Conclusion:

Join Algorithm	Approximate Cost
Nested-Loop	5050 I/O
Sort-Merge	450 I/O
Hash Join	450 I/O

Therefore, **Hash Join or Sort-Merge Join** is preferred over Nested-Loop Join. For an equality join, **Hash Join** is generally the best choice.

3.Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries using PreparedStatement.

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/company";
        String username = "root";
        String password = "root";

        String sql = "SELECT * FROM Employee WHERE dept_id = ?";

        try {
            // Establish connection
            Connection con = DriverManager.getConnection(
                url, username, password);

            // Create PreparedStatement
            PreparedStatement ps = con.prepareStatement(sql);

            // Set parameter
            ps.setInt(1, 10);

            // Execute query
            ResultSet rs = ps.executeQuery();

            // Display results
            while (rs.next()) {
                System.out.println(
                    rs.getInt("emp_id") + " " +
                    rs.getString("emp_name")
                );
            }

            // Close resources
            rs.close();
            ps.close();
            con.close();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

1. DriverManager.getConnection() establishes the database connection.
2. PreparedStatement creates a parameterized SQL query.
3. setInt() supplies the value for ?.
4. executeQuery() executes the SELECT statement.
5. ResultSet stores the returned rows.
6. Resources are closed after execution.

Advantage: PreparedStatement improves security by helping prevent **SQL injection** and can improve performance when the same query is executed repeatedly.

```
Directory: C:\Users\SREE\OneDrive\Desktop\CSA0506-Database Management System\JDBCProject

Mode                LastWriteTime         Length Name
----                -
-a---1            26-08-2026   09:52 PM         2613206 mysql-connector-j-26.7.0.jar

PS C:\Users\javac -cp ".;mysql-connector-j-26.7.0.jar" JDBCExample.java;C:\Users\javac -cp ".;mysql-connector-j-26.7.0.jar" JDBCExample.java
PS C:\Users\SREE\OneDrive\Desktop\CSA0506-Database Management System\JDBCProject> java -cp ".;mysql-connector-j-26.7.0.jar" JDBCExample
Database connected successfully!
101 Arun 10
103 Rahul 10
PS C:\Users\SREE\OneDrive\Desktop\CSA0506-Database Management System\JDBCProject>
```

4. Given the following schedule of four transactions, determine whether it is conflict-serializable using a precedence graph.

$S = R1(A) W2(A) R3(B) W1(B) R4(A) W3(B) W4(C) R2(C)$

Answer:

We construct a **precedence graph**.

Transactions:

$T1, T2, T3, T4$

Step 1: Find conflicts

Conflicting operations occur when:

- They belong to different transactions.
- They access the same data item.
- At least one operation is a write.

For A:

$R1(A) \rightarrow W2(A)$

Therefore:

$T1 \rightarrow T2$

$W2(A) \rightarrow R4(A)$

Therefore:

$T2 \rightarrow T4$

For B:

$R3(B) \rightarrow W1(B)$

Therefore:

$T3 \rightarrow T1$

$W1(B) \rightarrow W3(B)$

Therefore:

$T1 \rightarrow T3$

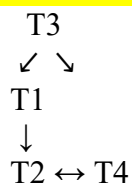
For C:

$W4(C) \rightarrow R2(C)$

Therefore:

$T4 \rightarrow T2$

Precedence graph



There is a cycle:

$T1 \rightarrow T3 \rightarrow T1$

and another:

$T2 \rightarrow T4 \rightarrow T2$

Conclusion:

Since the precedence graph contains cycles, the schedule is: NOT conflict-serializable

5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Consider:

T1: Lock-X(A)

Read(A)

Lock-X(B)

Write(B)

Unlock(A)

Unlock(B)

T2: Lock-X(B)

Read(B)

Lock-X(A)

Write(A)

Unlock(B)

Unlock(A)

Answer:

Initially:

T1 obtains X-lock(A)

T2 obtains X-lock(B)

Then:

T1 requests X-lock(B)

But B is already locked by T2.

Therefore:

T1 waits for T2

Next:

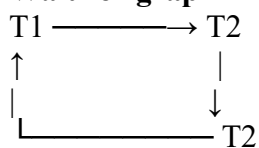
T2 requests X-lock(A)

But A is already locked by T1.

Therefore:

T2 waits for T1

Wait-for graph



More clearly:

T1 → T2

T2 → T1

There is a cycle.

Therefore, a **deadlock** exists.

Deadlock resolution

The DBMS can abort one transaction, for example T2.

Abort T2

↓

Release B

↓

T1 obtains B

↓

T1 completes

↓

Restart T2

Conclusion:

Basic 2PL guarantees conflict serializability, but it **does not prevent deadlocks**.

6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes.

Consider:

T1 timestamp = 5

T2 timestamp = 10

Suppose T1 requests a lock held by T2.

Answer:

Smaller timestamp means **older transaction**.

Therefore:

T1 = older

T2 = younger

Wait-Die Scheme

Rule:

- Older transaction requests lock held by younger → **Wait**
- Younger transaction requests lock held by older → **Die/Abort**

Here:

T1 → requests lock held by T2

T1 is older.

Therefore:

T1 waits for T2

Result:

T1 waits

Wound-Wait Scheme

Rule:

- Older transaction requests lock held by younger → **Wound/Abort younger**
- Younger transaction requests lock held by older → **Wait**

Here:

T1 → requests lock held by T2

T1 is older.

Therefore:

T1 wounds T2

So:

Abort T2

Release T2 locks

T1 obtains the requested lock

Result:

T2 is aborted

Comparison

Scheme	Older → Younger	Younger → Older
Wait-Die	Wait	Abort
Wound-Wait	Abort younger	Wait

Both schemes prevent **deadlocks** because circular waiting is avoided.

7. Given the following transaction log with a checkpoint, apply the Immediate Update recovery protocol after a system failure.

<START T1>
<T1, A, 100, 150>
<START T2>
<T2, B, 200, 250>
<COMMIT T1>
<CHECKPOINT>
<T2, C, 300, 350>
<START T3>
<T3, D, 400, 500>
<COMMIT T3>
--- SYSTEM CRASH ---

Answer:

Immediate Update allows database changes to be written to disk **before the transaction commits**.

After the crash, identify:

Committed transactions

T1

T3

These transactions must be **REDO**.

Uncommitted transaction

T2

T2 did not commit, so its changes must be **UNDO**.

Recovery operations

REDO T1

A: 100 → 150

REDO T3

D: 400 → 500

UNDO T2

Undo:

B: 250 → 200

C: 350 → 300

Final result

T1 → REDO

T2 → UNDO

T3 → REDO

Therefore:

REDO committed transactions and UNDO uncommitted transactions

8. Apply Deferred Update (NO-UNDO/REDO) recovery to the following log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<T2, C, 300, 350>

<START T3>

<T3, D, 400, 500>

<COMMIT T3>

--- SYSTEM CRASH ---

Answer:

In deferred update:

Database changes are not written to the database until the transaction commits.

Therefore, **UNDO is not required.**

After the crash:

Committed transactions

T1

T3

These transactions are REDO candidates.

Uncommitted transaction

T2

T2 is ignored.

Recovery

REDO T1:

A = 150

REDO T3:

D = 500

Ignore T2:

B = unchanged

C = unchanged

Final recovery table

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	Ignore
T3	Committed	REDO

Thus:

Deferred Update uses NO-UNDO/REDO

9. Design a concurrency control mechanism using Strict 2PL for a banking system where T1 transfers funds and T2 reads the balance simultaneously.

Assume:

T1: Transfer ₹100 from Account A to Account B

T2: Read balance of Account A

Answer:

Transaction T1

LOCK-X(A)

LOCK-X(B)

Read(A)

$A = A - 100$

Read(B)

$B = B + 100$

Write(A)

Write(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

Transaction T2

LOCK-S(A)

Read(A)

COMMIT

UNLOCK(A)

Strict 2PL execution

Suppose T1 executes first:

T1: X-lock(A)

T1: X-lock(B)

T1: $A = A - 100$

T1: $B = B + 100$

T1: COMMIT

T1: Unlock(A), Unlock(B)

T2: S-lock(A)

T2: Read(A)

T2: COMMIT

T2: Unlock(A)

T2 cannot read A while T1 has an exclusive lock.

Therefore, T2 waits until T1 commits.

Advantage

This prevents T2 from reading an intermediate state where money has been deducted from A but not yet added to B.

Thus Strict 2PL provides:

- Conflict serializability
- No dirty reads
- No cascading rollback
- Consistent banking transactions

10. Write pseudocode for query optimization using the heuristics:

- 1. Push down selection**
- 2. Project early**
- 3. Order joins by smaller intermediate results**

Answer:

Algorithm Heuristic_Query_Optimization(Query)

Input:

Relational algebra query tree

Output:

Optimized query tree

BEGIN

Step 1: Convert SQL query into
relational algebra/query tree

Step 2: Push selection operations down
toward the base relations

FOR each selection σ in query tree
 IF selection condition applies to a single relation
 Move σ below the join
 END IF
END FOR

Step 3: Push projection operations down

FOR each projection π
 Keep only attributes required by:
 - selection conditions
 - join conditions
 - final output
 Move π toward base relations
END FOR

Step 4: Determine join sizes

FOR each possible join
 Estimate size of intermediate result
END FOR

Step 5: Order joins

Select the join producing the smallest
intermediate relation first

Repeat until all joins are ordered

Step 6: Generate optimized query tree

Step 7: Return optimized query tree

END

Example

Original:

π A.name, B.salary
|
 σ B.salary > 50000
|
JOIN
/ \
A B

After pushing selection:

π A.name, B.salary
|
JOIN
/ \
A σ salary>50000
|
B

Instead of joining all tuples of B, only employees whose salary is greater than 50000 participate in the join.

Heuristic rules summary

Rule	Purpose
Push selection down	Reduce tuples early
Project early	Reduce attributes/tuple size
Join smaller relations first	Reduce intermediate results
Combine selections	Reduce number of operations
Avoid unnecessary operations	Reduce execution cost

Therefore, heuristic optimization attempts to produce a **lower-cost query execution plan without exhaustively evaluating every possible plan.**